you saw in figure 23.5, and it's rendered in the default Razor layout, _Layout.cshtml.

If you look inside _LoginPartial.cshtml, you can see how routing works with areas by combining the Razor Page path with an `{area}` route parameter using Tag Helpers. For example, the Login link specifies that the Razor Page `/Account/Login` is in the `Identity` area using the `asp-area` attribute:

```
<a asp-area="Identity" asp-page="/Account/Login">Login</a>
```

TIP You can reference Razor Pages in the `Identity` area by setting the `area` route value to `Identity`. You can use the `asp-area` attribute in Tag Helpers that generate links.

In addition to viewing the new files included thanks to ASP.NET Core Identity, open Program.cs and look at the changes there. The most obvious change is the additional configuration, which adds all the services Identity requires, as shown in the following listing.

Listing 23.1 Adding ASP.NET Core Identity services to `ConfigureServices`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

string connectionString = builder.Configuration                    ❶
    .GetConnectionString("DefaultConnection");                     ❶
builder.Services.AddDbContext<ApplicationDbContext>(options =>     ❶
    options.UseSqlServer(connectionString));                       ❶

builder.Services.AddDatabaseDeveloperPageExceptionFilter();        ❷

builder.Services.AddDefaultIdentity<IdentityUser>(options =>       ❸
    options.SignIn.RequireConfirmedAccount = true)                 ❹
        .AddEntityFrameworkStores<ApplicationDbContext>();         ❺
builder.Services.AddRazorPages();

// remaining configuration not show
```

❶ ASP.NET Core Identity uses EF Core, so it includes the standard EF Core configuration.